To stop shadows from being detected as lakes, you should **fuse Ascending and Descending orbits** to "fill in" dark spots, and use a **Slope Mask** (Slope < 5°) to filter out dark areas on mountainsides.

Some steep north-facing slopes are in shadow during *both* ascending and descending passes. Fusing them won't fix it because both values are "dark."

- **Solution:** You must aggressively mask these areas using a **Shadow Map** derived from the DEM and satellite geometry, not just a simple slope threshold.

2. Advanced Shadow Masking Code
Instead of just slope > 5, use the ee.Terrain.hillshade algorithm to simulate exactly where shadows fall for the specific time of year.

To classify glaciers, glacial lakes, and non-glacier areas from 20 Sentinel-1 images in Python, the best approach is using the geemap and ee (Google Earth Engine) libraries. This allows you to process the large multi-temporal dataset on the cloud, which is much faster than downloading

and processing 20 SAR scenes locally.
MDPI
MDPI
 +1
1. Multi-Temporal Python Code (GEE API)
The following code creates a Median Composite to automatically reduce shadows and speckle noise, then adds Slope and Elevation as key features for the Random Forest classifier.
MDPI
MDPI
 +2
python
```
import ee
import geemap

# 1. Initialize and define area of interest
ee.Initialize()
roi = ee.Geometry.Point([longitude, latitude]).buffer(10000)

# 2. Filter 20 images and create a Median Composite
s1_col = (ee.ImageCollection('COPERNICUS/S1_GRD')
          .filterBounds(roi)
          .filterDate('2023-06-01', '2023-09-30')
          .filter(ee.Filter.eq('instrumentMode',
```

```
'IW'))))

composite = s1_col.median().clip(roi)

# 3. Feature Engineering: Add Ratio, Slope,
and Elevation
ratio =
composite.select('VH').divide(composite.s
elect('VV')).rename('VH_VV_ratio')
dem = ee.Image('USGS/
SRTMGL1_003').clip(roi)
slope =
ee.Terrain.slope(dem).rename('slope')
elevation = dem.rename('elevation')

# This is your final multi-band input image
input_img = composite.addBands([ratio,
slope, elevation])

# 4. Supervised Classification (Random
Forest)
# Assuming 'training_pts' is a
FeatureCollection with a 'label' property
training = input_img.sampleRegions(
    collection=training_pts,
    properties=['label'],
    scale=10
)

classifier =
```

```
ee.Classifier.smileRandomForest(100).train
(
    features=training,
    classProperty='label',
    inputProperties=['VV', 'VH', 'VH_VV_ratio',
'slope', 'elevation']
)

classified = input_img.classify(classifier)
```
Use code with caution.

2. Local Processing with scikit-learn
If you already have the 20 images downloaded as GeoTIFFs, you can use scikit-learn to perform the classification locally.
MDPI
MDPI
python
```
import numpy as np
from sklearn.ensemble import
RandomForestClassifier
import rasterio

# Load processed SAR bands (after
calibration and terrain correction)
with rasterio.open('vv_median.tif') as src:
    vv = src.read(1)
with rasterio.open('slope.tif') as src:
    slope = src.read(1)
```

```
# Stack and reshape for the classifier
X = np.stack([vv.ravel(), slope.ravel()],
axis=1)

# Train and Predict (example labels)
clf =
RandomForestClassifier(n_estimators=100
)
clf.fit(X_train, y_train)
result = clf.predict(X).reshape(vv.shape)
```
Use code with caution.

How this works:

Median Composite: Averaging 20 images effectively "fills in" transient radar shadows and smooths out the grainy speckle noise characteristic of SAR.

Feature Integration: Adding Slope is mandatory for your use case; it allows the model to distinguish between "dark" lakes (flat) and "dark" shadows (steep).

Random Forest: This algorithm handles the non-linear relationship between backscatter and topography better than simple thresholding.

Key Details for Success:

Check Task Status: After running the script, go to the Google Earth Engine Tasks

tab (if using the Code Editor) or use task_raster.status() in Python to monitor progress.
Scale: Set the scale to 10 to match the native resolution of Sentinel-1.
CRS: By default, these exports use EPSG:4326 (WGS 84), which is the standard for GeoJSON.
Post-Processing: If your GeoJSON is too large, you can use a tool like Mapshaper to simplify the polygons without losing the general shape of the lakes.

Answer
To export, use ee.batch.Export.image.toDrive for the GeoTIFF raster and ee.batch.Export.table.toDrive (with fileFormat='GeoJSON') after using reduceToVectors to create the GeoJSON polygons.